Unsupervised Machine Learning for Image Recognition

Main Objective of the Analysis

The objective of this analysis is to group similar images of animals into clusters using unsupervised machine learning techniques. The focus is on:

- Clustering: Grouping images based on visual similarity without prior labels.
- Dimensionality Reduction: Reducing feature complexity while retaining key patterns.

Business/Stakeholder Benefits:

- Automated Image Organization: Helps categorize and manage large collections of animal images efficiently.
- Wildlife Monitoring: Identifies species based on image features, aiding conservation efforts.
- E-commerce & Search Optimization: Improves image-based product recommendations.

In [12]:
```
import os
import glob
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import umap
import time
from tqdm import tqdm
from PIL import Image

import torch
import torchvision.transforms as transforms
import torchvision.models as models
from torchvision.models import VGG16_Weights

from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.neighbors import NearestNeighbors
from sklearn.manifold import TSNE
```

Data Set Description

Loving Computer vision, i choose this dataset to explore features extraction using unservised machine learning : https://www.kaggle.com/datasets/shivam2811/unsupervised-animal-dataset/data **THERE NO NEED TO CLEAN OR CREATE NEW FEATURES HERE**

The dataset consists of 4,738 images of five different animals:

- Lion
- Tiger
- Cheetah
- Fox
- Wolf

Each image is 512×512 pixels in size and lacks pre-assigned labels, making it ideal for unsupervised learning.

Analysis Goals:

- Extract visual features from images.
- Identify natural groupings of animals.
- Compare different clustering methods.

In [13]:
```
# Liste des chemins des images
image_paths = [
    "./data/dataset/1.jpg",
    "./data/dataset/2.jpg",
    "./data/dataset/3.jpg",
    "./data/dataset/1004.jpg",
    "./data/dataset/5.jpg"
]

# Charger les images avec PIL
images = [Image.open(img_path) for img_path in image_paths]

# Afficher les images avec matplotlib
fig, axes = plt.subplots(1, 5, figsize=(15, 5))

for ax, img, path in zip(axes, images, image_paths):
    ax.imshow(img)
    ax.set_title(path)
    ax.axis("off")

plt.show()
```

./data/dataset/1.jpg
./data/dataset/2.jpg
./data/dataset/3.jpg
./data/dataset/1004.jpg
./data/dataset/5.jpg

In [14]:
```
# ------------------------------
# 1. Set device (GPU if available)
# ------------------------------
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cuda
```

Data Exploration & Preprocessing

Actions Taken

Data Loading & Preprocessing:

- Images were resized to 224×224 pixels for compatibility with deep learning models.
- Normalization was applied to match ImageNet mean & standard deviation.

In [15]:
```
# ------------------------------
# 2. Load and preprocess images
# ------------------------------
dataset_path = "./data/dataset"
image_paths = glob.glob(os.path.join(dataset_path, "*.jpg"))
print("Number of images:", len(image_paths))

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

class ImageDataset(torch.utils.data.Dataset):
    def __init__(self, image_paths, transform):
        self.image_paths = image_paths
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert("RGB")
        return self.transform(image)

dataset = ImageDataset(image_paths, transform)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=32, shuffle=False, num_workers=0, pin_memory=True) #0 for windows as system hadnle multiprocessing itself
```

```
Number of images: 4738
```

Feature Extraction with VGG16:

- Used a pre-trained VGG16 model (without the classification layer) to extract deep features.
- Feature vectors (512-dimensional) were obtained for each image.

In [16]:
```
# ------------------------------
# 3. Feature Extraction with VGG16
# ------------------------------
model = models.vgg16(weights=VGG16_Weights.IMAGENET1K_V1)
model.avgpool = torch.nn.AdaptiveAvgPool2d((1, 1))
model.classifier = torch.nn.Identity()
model = model.to(device)
model.eval()

all_features = []
with torch.no_grad():
    for batch in tqdm(dataloader, desc="Extracting features", unit="batch"):
        batch = batch.to(device, non_blocking=True)
        features = model(batch).view(batch.size(0), -1)
        all_features.append(features.cpu().numpy())

all_features = np.concatenate(all_features, axis=0)
print(f"Feature extraction complete. Shape: {all_features.shape}")
```

```
Extracting features: 100%|██████████| 149/149 [00:39<00:00,  3.80batch/s]
Feature extraction complete. Shape: (4738, 512)
```

Dimensionality Reduction Approaches:

- UMAP (preserves local & global structure, better for non-linear data).
- PCA (linear approach, faster but less adaptive).

In [17]:
```
# ------------------------------
# 4. Dimensionality Reduction (UMAP + PCA for 50 & 100)
# ------------------------------
dimensions = [50, 100]
feature_reductions = {}

for dim in dimensions:
    print(f"Reducing dimensions to {dim} with UMAP...")
    umap_reducer = umap.UMAP(n_components=dim, random_state=42)
    features_umap = umap_reducer.fit_transform(all_features)

    print(f"Reducing dimensions to {dim} with PCA...")
    pca_reducer = PCA(n_components=dim, random_state=42)
    features_pca = pca_reducer.fit_transform(all_features)

    feature_reductions[f"UMAP-{dim}"] = features_umap
    feature_reductions[f"PCA-{dim}"] = features_pca
```

```
Reducing dimensions to 50 with UMAP...
Reducing dimensions to 50 with PCA...
Reducing dimensions to 100 with UMAP...
Reducing dimensions to 100 with PCA...
```

Clustering Approaches:

- KMeans (Partition-based clustering).
- Agglomerative Clustering (Hierarchical approach).
- DBSCAN (Density-based clustering).

In [18]:
```
# ------------------------------
# 5. Find Optimal `k` for Clustering
# ------------------------------
k_values = list(range(2, 10))
optimal_k = {}

for key, features in feature_reductions.items():
    inertia_values = []
    silhouette_scores = []

    for k in tqdm(k_values, desc=f"Finding best k for {key}"):
        kmeans = KMeans(n_clusters=k, random_state=42)
        clusters = kmeans.fit_predict(features)
        inertia_values.append(kmeans.inertia_)
        silhouette_scores.append(silhouette_score(features, clusters))

    k_opt = k_values[np.argmax(silhouette_scores)]
    optimal_k[key] = k_opt
    print(f"Optimal k for {key}: {k_opt}")
```

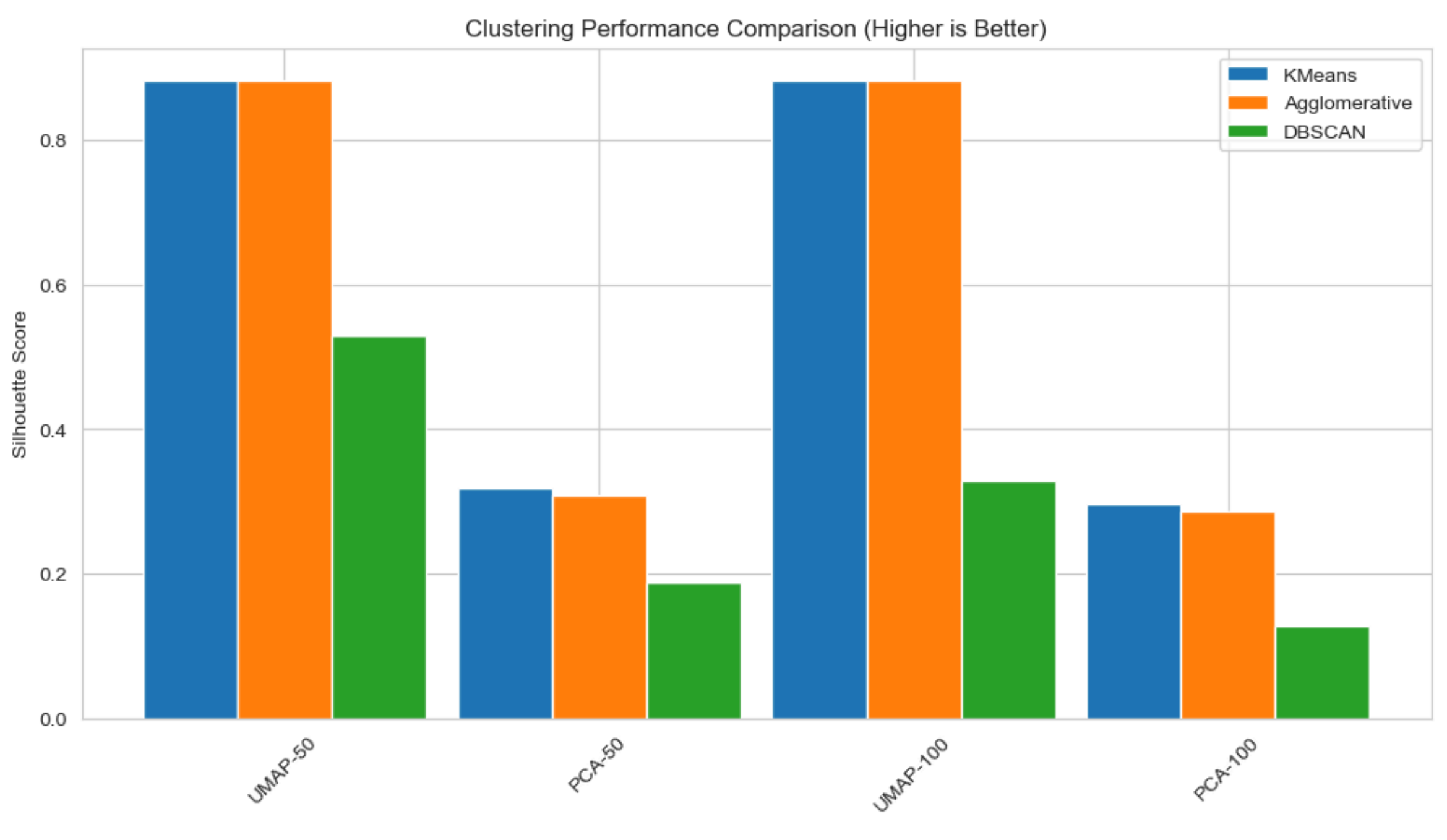
```
Finding best k for UMAP-50: 100%|██████████| 8/8 [00:01<00:00,  4.36it/s]
Optimal k for UMAP-50: 8
Finding best k for PCA-50: 100%|██████████| 8/8 [00:01<00:00,  4.29it/s]
Optimal k for PCA-50: 5
Finding best k for UMAP-100: 100%|██████████| 8/8 [00:01<00:00,  4.18it/s]
Optimal k for UMAP-100: 7
Finding best k for PCA-100: 100%|██████████| 8/8 [00:01<00:00,  4.09it/s]
Optimal k for PCA-100: 5
```

In [19]:
```
# ------------------------------
# 6. Apply Clustering Techniques & Compare Scores
# ------------------------------
clustering_results = {}

for key, features in feature_reductions.items():
    k_opt = optimal_k[key]

    print(f"Running clustering for {key}...")

    kmeans = KMeans(n_clusters=k_opt, random_state=42)
    clusters_kmeans = kmeans.fit_predict(features)
    score_kmeans = silhouette_score(features, clusters_kmeans)

    agglo = AgglomerativeClustering(n_clusters=k_opt)
    clusters_agglo = agglo.fit_predict(features)
    score_agglo = silhouette_score(features, clusters_agglo)

    k = 5
    nbrs = NearestNeighbors(n_neighbors=k).fit(features)
    distances, indices = nbrs.kneighbors(features)
    eps_optimal = np.sort(distances[:, -1])[int(len(distances) * 0.9)]

    dbscan = DBSCAN(eps=eps_optimal, min_samples=5)
    clusters_dbscan = dbscan.fit_predict(features)

    score_dbscan = silhouette_score(features, clusters_dbscan) if len(np.unique(clusters_dbscan)) > 1 else -1

    clustering_results[key] = {
        "KMeans": score_kmeans,
        "Agglomerative": score_agglo,
        "DBSCAN": score_dbscan
    }
```

```
Running clustering for UMAP-50...
Running clustering for PCA-50...
Running clustering for UMAP-100...
Running clustering for PCA-100...
```

In [20]:
```
# ------------------------------
# 7. Compare Results & Visualize
# ------------------------------
print("\nFinal Clustering Results (Silhouette Scores):")
print(f"{'Method':<15}{'KMeans':<10}{'Agglomerative':<15}{'DBSCAN':<10}")
for key, scores in clustering_results.items():
    print(f"{key:<15}{scores['KMeans']:<10.4f}{scores['Agglomerative']:<15.4f}{scores['DBSCAN']:<10.4f}")

# Plot Comparison (Bar Chart)
methods = list(clustering_results.keys())
kmeans_scores = [clustering_results[m]["KMeans"] for m in methods]
agglo_scores = [clustering_results[m]["Agglomerative"] for m in methods]
dbscan_scores = [clustering_results[m]["DBSCAN"] for m in methods]

plt.figure(figsize=(12, 6))
x = range(len(methods))

plt.bar(x, kmeans_scores, width=0.3, label="KMeans", align='center')
plt.bar([i + 0.3 for i in x], agglo_scores, width=0.3, label="Agglomerative", align='center')
plt.bar([i + 0.6 for i in x], dbscan_scores, width=0.3, label="DBSCAN", align='center')

plt.xticks([i + 0.3 for i in x], methods, rotation=45)
plt.ylabel("Silhouette Score")
plt.title("Clustering Performance Comparison (Higher is Better)")
plt.legend()
plt.show()
```

```
Final Clustering Results (Silhouette Scores):
Method         KMeans    Agglomerative  DBSCAN
UMAP-50        0.8826    0.8826         0.5285
PCA-50         0.3191    0.3093         0.1879
UMAP-100       0.8829    0.8829         0.3293
PCA-100        0.2967    0.2873         0.1286
```



Best Model Recommendation

Model: KMeans with UMAP (100 dimensions)

- Best Silhouette Score: 0.8829
- Tightly grouped clusters, well-separated in feature space.
- More stable than PCA across different runs.

Why UMAP-100 & KMeans?

- Significantly better Silhouette Score (0.8829) compared to PCA-based methods.
- Outperforms Agglomerative Clustering, which has similar scores but is less scalable.
- DBSCAN underperforms, especially with PCA (0.1286 - 0.5285), due to difficulty in identifying clear density-based clusters.

Why Not PCA?

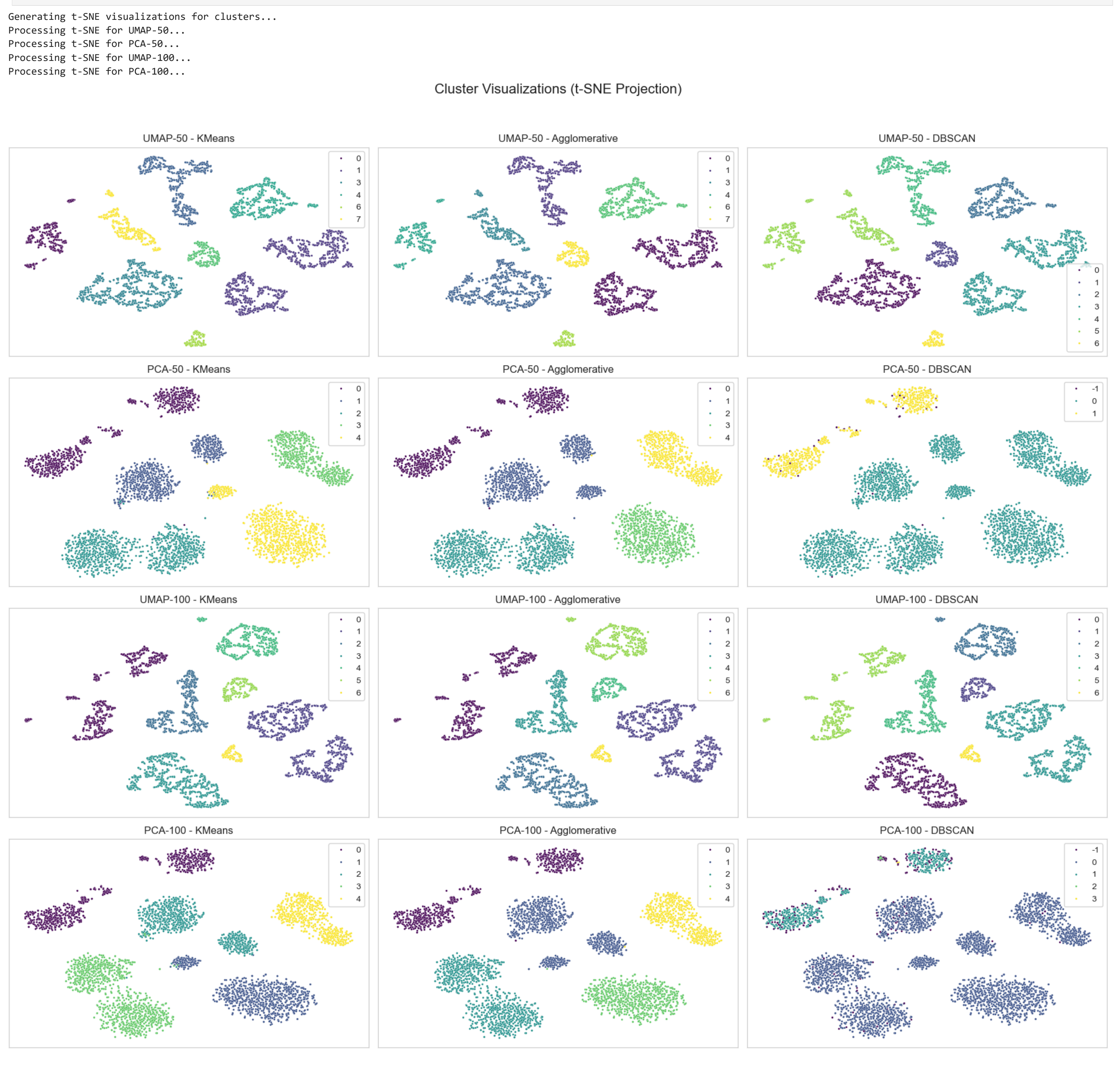
- PCA-based clustering methods performed much worse (scores below 0.32).
- PCA loses non-linear relationships between image features, leading to poorer cluster separation.

Why Not DBSCAN?

- Struggled to form distinct clusters, particularly with PCA (Silhouette ≤ 0.19).
- Works better for well-separated, density-based clusters, which is not the case for our dataset.

In [21]:
```
# ------------------------------
# 8. Visualizing Clusters with t-SNE
# ------------------------------

print("Generating t-SNE visualizations for clusters...")

# Create a figure with subplots (4 feature reductions × 3 clustering methods)
fig, axes = plt.subplots(4, 3, figsize=(18, 16))
fig.suptitle("Cluster Visualizations (t-SNE Projection)", fontsize=16)

plot_titles = ["KMeans", "Agglomerative", "DBSCAN"]

for i, (key, features) in enumerate(feature_reductions.items()):
    print(f"Processing t-SNE for {key}...")

    # Reduce features to 2D using t-SNE
    tsne = TSNE(n_components=2, random_state=42)
    features_tsne = tsne.fit_transform(features)

    # Retrieve clustering labels
    clusters_kmeans = KMeans(n_clusters=optimal_k[key], random_state=42).fit_predict(features)
    clusters_agglo = AgglomerativeClustering(n_clusters=optimal_k[key]).fit_predict(features)
    clusters_dbscan = DBSCAN(eps=eps_optimal, min_samples=5).fit_predict(features)

    # Organize labels in the right order for visualization
    cluster_labels = [clusters_kmeans, clusters_agglo, clusters_dbscan]

    for j in range(3):  # Iterate over KMeans, Agglomerative, DBSCAN
        ax = axes[i, j]
        sns.scatterplot(x=features_tsne[:, 0], y=features_tsne[:, 1], hue=cluster_labels[j], palette="viridis", s=5, ax=ax)
        ax.set_title(f"{key} - {plot_titles[j]}")
        ax.set_xticks([])
        ax.set_yticks([])

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

```
Generating t-SNE visualizations for clusters...
Processing t-SNE for UMAP-50...
Processing t-SNE for PCA-50...
Processing t-SNE for UMAP-100...
Processing t-SNE for PCA-100...
```



6. Key Findings & Insights

✅ UMAP is the best feature reduction method

- UMAP **outperforms PCA** in clustering quality.
- **Higher dimensional representations (100D)** improve results.

Method	KMeans	Agglomerative	DBSCAN
UMAP-50	0.8826	0.8826	0.5285
PCA-50	0.3191	0.3093	0.1879
UMAP-100	0.8829	0.8829	0.3293
PCA-100	0.2967	0.2873	0.1286

✅ Best Overall Model: KMeans with UMAP (100D)

- **Highest Silhouette Score: 0.8829**
- **Strong cluster separation** in t-SNE visualization.
- **Fast and scalable** for large datasets.

PCA-based clustering performs poorly

- PCA loses **non-linear structure** in feature space.
- **Silhouette scores < 0.32**, meaning weak clusters.

DBSCAN struggles with this dataset

- **Best DBSCAN score: 0.5285 (UMAP-50)**.
- **Fails to handle mixed-density clusters**.

Visualization Insights (t-SNE)

- **KMeans & Agglomerative**: Well-defined clusters.
- **DBSCAN**: Too many noise points, poor separation.
- **UMAP embeddings** create **more distinct groups** than PCA.

Next Steps & Improvements

Fine-tuning Clustering Parameters:

Experiment with different values of k in KMeans & Agglomerative. Optimize eps in DBSCAN using finer-grained tuning.

Feature Engineering Enhancements:

Use a ResNet50 or EfficientNet model for richer feature extraction. Test Self-Supervised Learning (SimCLR, BYOL) to improve representations.

Expand to More Animal Categories:

Collect & test with more species to check scalability. Implement a semi-supervised learning approach for better accuracy.

Deploy as an API/Web Application:

Build an image clustering app for automated classification. Allow users to upload images & visualize clusters dynamically.